Clockwork Dash: Milestone 3 Report

CSC 581 · Smruthi Bangalore Thandava Murthy · Unity ID: sbangal6 · October 21, 2026

1. Engine Design: the Game Object Model

We designed the engine around composition so that new objects could share behaviour without inheriting from a growing class hierarchy. A `GameObject` holds its identity and components, while systems implement behaviour using the data in those components. I used this design to build Clockwork Dash, a four-player time-trial platformer with moving platforms, hazards, checkpoints and temporary player effects. The same object definitions work across the server, clients and peers.

1.1 How the model works

Each `GameObject` has an id, a replicated kind such as `player` or `mover`, and a local name for debugging. Its components are stored in a small vector, with a flag recording whether each component came from the network. We need that distinction when applying snapshots because removing an outdated replicated component should not delete components that a client attached locally.

For component lookup, we chose per-type keys instead of runtime type information. `KeyOf<T>()` returns the address of a per-type static value, and `Get<T>()` and `Has<T>()` compare these keys while scanning an object's slots. The lookup is still a linear search, but each type check is a pointer comparison.

The component interface defines `TypeName()`, `Write(Writer&)` and `Read(Reader&)`. This separates the fields a component contains from how those fields are encoded. I can describe a game component once and use either the text or binary writer with it. Components without a network name, such as `Camera`, stay local.

`World` owns the objects in an id-ordered map, and `Each<Ts...>()` selects those with the requested components. We defer destruction until `Flush()` so systems can finish iterating safely. Separate id ranges prevent locally created objects from colliding with replicated ones. Server ids start at 1, client-local ids at 2^30, and peer n uses a range starting at (n + 1) × 2^24.

We register networked components in `ComponentRegistry`, which maps each name to a factory and a 16-bit wire identifier. Registration checks for identifier collisions and stops startup if it finds one. The receiver can then construct a component from its wire identifier without needing game-specific logic in the decoder.

The engine provides 13 components: `Transform`, `RigidBody`, `Collider`, `Renderable`, `InputState`, `PlatformerController`, `PathMover`, `SpawnPoint`, `DeathZone`, `Respawn`, `Camera`, `ScrollBoundary` and `NetIdentity`. I built the game on their movement, physics, spawning, camera and rendering systems. Authority filters allow the same simulation code to update all players on a server or only the local player's objects on a peer.

1.2 Alternatives, and why we chose composition

Design	Why we did not choose it
An inheritance tree, such as Entity → Platform → MovingPlatform.	The assignment rules out this approach. It also makes combinations awkward. A saw needs both movement and hazard behaviour, while a potion ledge needs both collision and pickup behaviour.
One large Entity class that holds every possible field.	Each new feature adds fields that many objects never use. Serialization must also distinguish relevant fields from unused ones.
A property bag with string keys.	It is flexible, but names and value types need runtime checks. Misspelled keys are harder to catch, and a generic format needs additional field identifiers.
A packed ECS with dense component arrays.	It could improve iteration and memory locality. It also requires more storage and query machinery than we wanted to introduce in this milestone.

A single `Entity` class is enough while every object is a coloured box, but Clockwork Dash has behaviours that overlap. A saw needs movement and damage, while a potion ledge needs collision and pickup logic. I could combine those features with components without adding another branch to an inheritance tree or more unused fields to every object.

We kept components with their object partly because that made replication straightforward. The encoder asks each registered component to write its fields, so a new game object does not need its own encoder branch. Adding a component requires a matching reader, writer and registration, while the transport remains the same.

Keeping an object's state together also suited how I needed to debug the game. A runner combines physics, race progress, ownership and temporary effects. Its id and readable kind give me a way to follow that state across processes without treating each component as an unrelated value.

For this milestone, I preferred the simpler storage model over the additional machinery of a packed ECS. That was a design tradeoff, not a measured performance result. The networking benchmark does not compare storage layouts.

1.3 Components and behaviours for the required objects

I represented hidden objects by leaving out `Renderable`. They still take part in collision, triggers and replication, which lets spawn points and death zones use the same object model as visible platforms. The table below shows how I assembled the required objects.

Required object	In Clockwork Dash	Components	Behaviour
Player character.	Each client has one robot runner.	Transform, RigidBody, InputState, PlatformerController, Renderable, NetIdentity, Respawn, Racer, Knockback and Boost.	Controllers turn buttons into movement. Physics resolves collisions. Game systems handle progress, hazards, potions and stomps.
Static platforms, with at least three variations.	There are nine ledges and four decks. Examples include teal 230 × 28, blue 160 × 40 and violet 140 × 22 platforms.	Transform, Renderable, a solid Collider and Impact.	These platforms block movement. Head bumps can also trigger a game-specific response.

Required object	In Clockwork Dash	Components	Behaviour
Moving platforms, with at least two patterns.	Five platforms use two horizontal paths, two vertical paths and one circular path.	Transform, Renderable, a one-way Collider and PathMover.	MovePaths computes position from time. Physics carries a standing rider with the platform.
Hidden spawn points.	Four starting slots and three checkpoint spawns are available.	Transform and SpawnPoint.	PlaceAtSpawn places a runner at its assigned spawn index.
Hidden death zone.	A trigger extends below the course.	Transform, a trigger Collider and DeathZone.	A falling runner returns to its current spawn.
Hidden scrolling boundaries.	Each local camera has two boundaries, inset by 560 units.	Camera is local. Boundary objects use Transform and ScrollBoundary.	UpdateCameras moves the view when the runner crosses a boundary.
Additional game objects.	The course has four saws, three barriers, three checkpoints, a finish gate and three potion ledges.	These use Hazard, Checkpoint, Finish and PotionPlatform alongside engine components.	Saws use circle-versus-box collision. Barriers cycle through safe, warning and live phases.

I added nine components for the game: `Racer`, `Checkpoint`, `Hazard`, `Finish`, `Impact`, `Knockback`, `PotionPlatform`, `Boost` and `PeerSession`. These hold the race-specific state while the engine continues to handle reusable behaviour. Figures A1 and A2 show the visible course and the hidden objects that support it.

1.4 What the design enables, and its limits

The main benefit is being able to reuse behaviour in new combinations. For example, my saw uses `PathMover` for movement and `Hazard` for damage. The same `Step` function runs in solo practice, on the server and on peers, with an authority filter selecting which runners it can update.

The registry also gives me a starting point for data-driven levels, save files and replay tools because it can construct components from their names. Those extensions would need validation, version handling and a policy for restoring runtime state. They would build on the existing object model rather than require a new one.

We accepted some limits to keep the implementation manageable. Components use separate allocations, queries scan the world, and collision detection has no spatial partition. In the game, I specify system order manually and must ensure that objects have the components each system expects. The world mutex protects shared state but limits parallel simulation. Readers and writers must agree on field order and protocol version. Moving platforms are one-way because the physics does not resolve crushing between solid surfaces.

2. Network Architecture

Clockwork Dash supports both client-server and peer-to-peer play. I kept the race logic shared between the two modes and changed which process is responsible for simulating each runner. In the default client-server mode, clients send button input to a headless server, which updates the entire race and broadcasts world snapshots. In P2P mode, each peer simulates its own runner and sends updates directly to the other peers. Section 3 compares four networking choices, including a client-server relay against direct peer-to-peer messaging.

2.1 Why client-server is the primary model

I chose client-server as the default because the race includes interactions that need a single outcome. If two runners reach the same potion, only one should receive the boost. Similarly, a stomp must affect the same player on every screen. Processing these interactions in one authoritative world lets the server resolve them once and distribute the result to all clients.

This is also why clients send `InputState` rather than their calculated positions or race progress. The server applies the buttons through the controller and physics systems, then updates checkpoints, deaths and finish times. Clients receive that state through snapshots. A full snapshot gives a late joiner the current race state without replaying earlier interactions.

The main tradeoff is responsiveness. Without client-side prediction, a player's movement becomes visible after the input reaches the server and an updated snapshot returns. The race also depends on the server remaining available. I kept this model as the default because it gives shared interactions a clear authority, while P2P provides a second way to distribute the simulation.

2.2 Peer-to-peer as a second mode

For P2P, I gave each peer authority over its own runner while retaining the same movement and race systems. Each peer binds a PUB socket on `portBase + 200 + id` and subscribes to the others. Peers exchange start times to establish a shared epoch, allowing moving-platform positions and hazard phases to be calculated locally from time.

Player ownership alone does not resolve shared interactions, so I use an event log for potion openings, potion claims, ledge impacts and stomps. Events are sent again every 100 ms and identified by owner and sequence number, allowing receivers to ignore duplicates. A deterministic ordering resolves competing potion claims so peers can agree on the same result.

This keeps the simulation distributed, but each peer must trust the others to report their runners correctly. It also makes time agreement part of the design. The current implementation assumes compatible clocks, so extending it across machines would require clock synchronization and tests with delayed events.

2.3 Threads and sockets

I used REQ/REP for joins and input requests, and PUB/SUB for world snapshots. Each client has a dedicated server request thread, so one client's exchange does not block the others. The server's main thread simulates the race, while each client's network worker handles communication separately from keyboard sampling and rendering. Figure 1 shows how these responsibilities fit together.

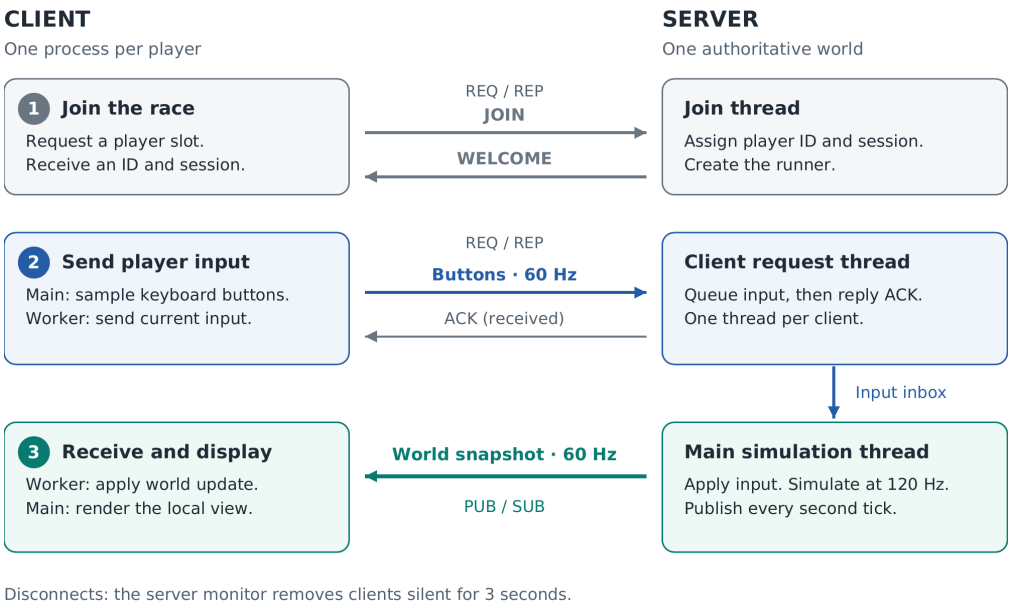


Figure 1. Client-server message flow. Joining happens once. Input and world updates repeat during play. ACK confirms receipt of input, while a later snapshot carries the result.

Request threads queue input under an inbox mutex and reply `ACK`. At the start of each simulation step, the main thread takes the queued input and applies it under the world mutex before updating the game. This makes ACK a confirmation of receipt, not of completed simulation. The server targets 120 steps per second and publishes after every second step, giving a nominal 60 world updates per second. Join and leave callbacks also use the world mutex when changing objects.

On the client, the worker sends input and receives snapshots while the main thread handles keyboard input, cameras and rendering. Both threads lock the local world when accessing component data, but release the lock before network waits. A delayed reply can therefore stall communication while rendering continues with the existing state. Each socket is used by only one thread at a time. I kept this separation to avoid making the rendering loop wait for a server response.

2.4 Joining, leaving and disconnects

A client sends `JOIN` to the server, which assigns a player slot and session number, creates the runner and replies with `WELCOME`. The reply includes the dedicated request port. The client connects to that port and subscribes to world updates. I force the next publication to be a keyframe, a full snapshot from which the client can build its world. Deltas are ignored until a keyframe arrives because changes alone cannot describe the starting state.

If no exchange succeeds for one second, the client sends `PING` and expects `PONG`. These heartbeats use wall time so pausing gameplay does not normally expire the connection. A `BYE` message or more than three seconds of silence triggers the same cleanup: remove the client's objects and snapshot history, then free its slot. Other clients receive the removal and continue racing, as Figures A10 and A11 show.

I use the session number to distinguish successive players assigned the same slot. Requests from an expired session receive `GONE`, preventing an old connection from controlling the new occupant's runner.

In P2P mode, each peer claims its id by binding a port. Receivers remove its objects on `BYE` or after three seconds of silence. Surviving peers retain the shared event history so later joiners can recover course changes made by players who have left. Figures A12 and A13 show a two-peer session

continuing after peer 0 exits.

2.5 Data format

When a player holds Right in client-server mode, the client sends the held buttons in `InputState`. This is a 32-bit integer in which each bit represents a button. The server uses that input to calculate movement, then sends the resulting player state back to the clients. In P2P mode, each peer calculates and sends its own runner's state.

I use a common message structure, called a **snapshot**, for these exchanges. Its contents depend on the sender. A client sends input, while the server sends world state such as positions and velocities. Only objects marked with `NetIdentity` and components registered for networking are included. Cameras stay local because each player needs a separate view. Input messages are marked partial so receiving buttons does not erase the runner's other components.

Each snapshot contains the following parts in order.

Part	Contents and purpose
Message header.	An encoding marker and version identify how to read the message. Flags identify full or delta updates and complete or partial component records. A counter, timestamp and object count describe the update.
Object records.	Each record identifies the object, its kind and owner, and its sample time. A component count is followed by component types and their values, such as position or button state.
Removal list.	A count and object ids tell the receiver which objects to delete.

Text and binary encode the same component values differently. Text stores readable, space-separated numbers and component names. Binary stores numbers directly, using little-endian integers, four-byte floats and two-byte component identifiers. Each component defines its field order for both encodings. I kept text for inspection and binary to avoid converting numbers to characters and repeating component names.

Full and delta determine which objects are included. A full snapshot includes every object selected by the sender. A delta includes new or changed objects and ids of removed objects. For example, a moving platform is included while an unchanged ledge is omitted. A changed object still sends all its selected components, not just the changed position. I detect changes by serializing and comparing component data, so smaller messages still require encoding work.

Delta mode sends a full snapshot, or keyframe, every 60 snapshots. This supplies a baseline for new clients and repairs stale state when received. Missing updates are not explicitly retransmitted. Either update mode can use text or binary, giving four combinations to compare.

3. Performance Analysis

I compared four data-exchange strategies. The two options in each pair differ in only one setting. Binary encoding and direct peer-to-peer messaging gave the largest gains: binary needed 15–57% less time than text, and P2P needed 30–69% less than a client-server relay. Sending button input instead of player state saved 5–13%, and downstream delta snapshots saved up to 28% when most objects were static.

3.1 Strategies compared

Each comparison changes one setting of the networking path and holds the others fixed, so any difference in completion time comes from that setting. Three of them match the examples in the assignment: the type of information sent, its representation and the network model. The fourth varies how much of the world each update carries. All four run on the engine's `World`, systems, snapshot encoder and decoder, and 0MQ socket wrapper. The message sizes below are for the baseline condition of 100 moving platforms, 100 static platforms and two players.

Comparison	A	B	Held fixed
Information	Player state: each client simulates its runner and sends its components (131 bytes)	Button input: each client sends its `InputState` (54 bytes) and the server simulates every runner	CS, binary, full snapshots
Representation	Text: decimal fields with component names (30.2 KB per world snapshot)	Binary: raw little-endian fields with 2-byte component ids (17.6 KB)	CS, button input, full snapshots
Update size	Full: every object in every world snapshot	Delta: new and changed objects plus removed ids, with a full keyframe every 60 snapshots	CS, button input, binary
Topology	CS relay: client to server over REQ/REP, then server to the other clients over PUB/SUB	P2P mesh: each peer publishes directly to every other peer	Player state, binary, full snapshots

3.2 Methodology

Each comparison has three experiments, one per sweep: 10, 100 and 500 moving platforms with 100 static platforms and two players; 10, 100 and 500 static platforms with 100 moving platforms and two players; and two, three and four players with 100 platforms of each type. Platforms are placed from a seed on a level with spawn points and a death zone, and the moving platforms follow horizontal, vertical and circular paths.

Each run simulates 500 warm-up ticks and then 100,000 measured ticks at a fixed step of 1/120 s, with no real-time cap, and publishes an update every second tick. The metric is the wall-clock time until every endpoint has completed its ticks and applied the final update. Runners follow seeded input traces, so both sides of a pair get the same level and the same inputs. Each condition ran ten seeds per side, five in AB order and five in BA order, for $4 \times 9 \times 2 \times 10 = 720$ runs. All runs used a Release build on an Apple M2 Pro (12 cores, 16 GB RAM, macOS 26.6.2) over localhost TCP.

Flow control keeps each sender at most four updates ahead of its slowest receiver, so no update is dropped or merged. At the start, after warm-up and at the end of each run, every endpoint's world is compared bit for bit with an offline simulation of the same seed. All 720 runs matched on the first attempt, so each strategy changes performance without changing gameplay. Tables 1–4 give the mean and sample variance of the ten runs per side, and the geometric mean of the ten paired ratios B / A with a 95% t-interval. Figures B1–B4 plot the times next to the data received, and Figure B5 plots the ratios.

3.3 Results: information and representation

Times are seconds for 100,000 ticks, variances are in s^2, and a ratio below 1 means B was faster. The 100-moving, 100-static and two-player rows have the same configuration but were measured independently, and their ratios agree to within 0.012.

Table 1. Player state (A) versus button input (B) in CS mode. Figure B1 shows time and data.

Condition	Player state mean (s)	Player state variance (s²)	Button input mean (s)	Button input variance (s²)	Ratio B / A [95% CI]
10 moving	11.105	0.0885	10.394	0.0913	0.936 [0.910, 0.962]
100 moving	11.942	0.0749	10.829	0.0778	0.907 [0.882, 0.933]
500 moving	19.319	0.0224	16.898	0.3238	0.874 [0.853, 0.896]
10 static	11.172	0.1059	10.323	0.0675	0.924 [0.905, 0.944]
100 static	12.038	0.0827	10.805	0.0745	0.898 [0.869, 0.927]
500 static	17.058	0.0486	14.899	0.2075	0.873 [0.851, 0.896]
2 players	11.946	0.0938	10.695	0.0882	0.895 [0.864, 0.927]
3 players	14.117	0.3912	13.259	0.2062	0.940 [0.919, 0.961]
4 players	15.597	0.0442	14.747	0.0287	0.946 [0.934, 0.958]

Button input was faster in all nine conditions, by 5–13%. The gain comes from removing client work, not from the smaller message. In state mode each client also simulates its own copy of the world, which costs 2.5 s of simulation across the two clients at the baseline and 7.9 s with 500 moving platforms. The server waits for every client's update before it publishes the next one, so this work is on the critical path. Data received changed by less than 1%, because the 17.6 KB world snapshot sent downstream outweighs the 131-byte and 54-byte upstream messages.

Table 2. Text (A) versus binary (B) encoding in CS mode. Figure B2 shows time and data.

Condition	Text mean (s)	Text variance (s²)	Binary mean (s)	Binary variance (s²)	Ratio B / A [95% CI]
10 moving	12.376	0.0887	10.481	0.1323	0.847 [0.824, 0.870]
100 moving	15.130	0.0712	10.775	0.0805	0.712 [0.696, 0.729]
500 moving	39.735	5.5342	17.308	0.2137	0.436 [0.415, 0.459]
10 static	12.691	0.0711	10.065	0.0635	0.793 [0.775, 0.811]
100 static	15.051	0.0802	10.682	0.0543	0.710 [0.698, 0.721]
500 static	34.529	18.2720	14.622	0.2956	0.426 [0.391, 0.465]
2 players	15.000	0.0596	10.670	0.1489	0.711 [0.691, 0.731]
3 players	17.252	0.0807	12.472	0.0230	0.723 [0.714, 0.732]
4 players	19.290	0.0311	14.653	0.0353	0.760 [0.751, 0.768]

Binary was faster in every condition, and its lead grows with the number of objects: it needed 15% less time with 10 moving platforms, 29% less at the baseline and 56–57% less with 500 platforms. At the baseline a binary snapshot is 42% smaller, the server encodes it in 41 µs instead of 105 µs, and a client decodes it in 28 µs instead of 110 µs. Binary copies fields directly, while text formats and parses every number and repeats each component name. Both costs grow with every object, so with 500 moving platforms decoding a text snapshot took 354 µs against 85 µs. Text with 500 static platforms was the most variable condition: three runs took about 28 s and seven took 36–38 s with the same server CPU time, so the extra time was spent waiting, not computing.

3.4 Results: update size and topology

Table 3. Full (A) versus downstream delta (B) snapshots in CS mode. Figure B3 shows time and data.

Condition	Full mean (s)	Full variance (s^2)	Delta mean (s)	Delta variance (s^2)	Ratio B / A [95% CI]
10 moving	9.837	0.0515	8.860	0.0522	0.901 [0.881, 0.920]
100 moving	10.723	0.1355	10.277	0.0674	0.959 [0.931, 0.987]
500 moving	16.935	0.1632	14.464	0.0402	0.854 [0.834, 0.875]
10 static	10.204	0.0834	10.172	0.1083	0.997 [0.960, 1.034]
100 static	10.538	0.1016	10.122	0.0454	0.961 [0.929, 0.993]
500 static	14.654	0.2974	10.597	0.0158	0.724 [0.705, 0.743]
2 players	10.617	0.0781	10.179	0.0269	0.959 [0.937, 0.981]
3 players	12.425	0.0146	12.079	0.0425	0.972 [0.963, 0.982]
4 players	14.684	0.0266	14.197	0.0163	0.967 [0.959, 0.975]

Delta was faster in eight of the nine conditions, and its benefit depends on how much of the world stays still. With 500 static platforms it cut the world snapshot from 46.0 KB to 10.7 KB and the run time by 28%. With 500 moving platforms the snapshot shrank only from 57.2 KB to 49.9 KB, because every moving platform is resent with all of its components, and the time fell by 15%. With 10 static platforms almost every object moves, and the ratio was 1.00. Delta also moves work from the clients to the server. Change detection serializes and compares every component, so encoding cost 10–30% more (49 µs against 41 µs at the baseline), while decoding on each client fell from 28 µs to 17 µs. The clients set the pace in CS mode, so this trade shortened the run even with 500 moving platforms, where total CPU time stayed the same.

Table 4. CS relay (A) versus direct P2P (B). Figure B4 shows time and data.

Condition	CS relay mean (s)	CS relay variance (s^2)	P2P mean (s)	P2P variance (s^2)	Ratio B / A [95% CI]
10 moving	9.370	0.0267	2.901	0.0197	0.309 [0.300, 0.319]
100 moving	10.064	0.0129	3.296	0.0109	0.327 [0.320, 0.335]
500 moving	12.655	0.0052	5.857	0.0465	0.463 [0.451, 0.474]
10 static	9.458	0.0098	2.937	0.0019	0.311 [0.306, 0.315]
100 static	9.912	0.0030	3.230	0.0003	0.326 [0.324, 0.328]
500 static	12.188	0.0587	5.164	0.0052	0.424 [0.419, 0.429]
2 players	9.944	0.0083	3.242	0.0018	0.326 [0.324, 0.328]
3 players	11.890	0.0054	5.238	0.0165	0.440 [0.433, 0.448]
4 players	15.698	0.0582	10.935	0.1141	0.696 [0.680, 0.713]

P2P was faster in every condition. It needed about one third of the relay's time with two players (ratio 0.33) and 0.70 of it with four. Both sides send the same 131-byte player-state messages and simulate the level locally, so the difference comes from the message path. A relay update takes two hops, and the first is a synchronous REQ/REP exchange in which the client blocks until the server replies. That exchange costs each client about 140–160 µs per update, while a P2P peer publishes once and never waits for a reply. The lead narrows with more players because the mesh grows quadratically. With four

peers, each one receives three streams and must wait for its slowest receiver, so flow-control waiting rose from 0.3 s to 17.1 s and data received rose from 0.43× to 0.78× of the relay's.

3.5 Conclusions

Binary encoding should be the default, because it was never slower and its lead grows with scene size. Delta snapshots pay off when most objects are static. In a scene dominated by moving objects they add encoding cost for little reduction in size, and dirty flags set by the systems would avoid serializing unchanged components. Button input was faster than player state and keeps the server authoritative over every runner. The largest fixed cost in CS mode is the synchronous REQ/REP upstream, which takes about two thirds of a baseline run. It narrows the ratios of the other three comparisons, which all run in CS mode. In absolute terms, text cost 4.4 s more than binary at the baseline and 22.4 s more with 500 moving platforms. An asynchronous upstream pattern such as DEALER/ROUTER is the clearest next optimization for the client-server path.

4. Reflection

4.1 What worked as expected

Composition worked well when I moved beyond the required platforms and player. A runner needed race progress and temporary effects, while a checkpoint needed only a region and an order. Keeping those differences in game components let me add rules without making the engine depend on the race. The nine additional components gave the design a practical test through several different kinds of objects.

Sharing `clockwork::Step` across solo, CS and P2P modes also paid off. Movement, hazards and race progress stayed in one implementation, with authority filters controlling which runners an endpoint could update. This kept changes to game rules in one place and made ownership the main question when checking an interaction across modes.

I expected binary to beat text, but the delta comparison taught me more. Change detection made delta encoding 10–30% more expensive, yet delta still finished sooner in eight of nine conditions, because sending and decoding less data more than paid for it. Measuring both sides of that tradeoff gave me a better explanation than simply calling one update format faster.

4.2 What did not

The component model still leaves too many dependencies to the caller. An object can have a component without another one that its system requires, and the game depends on the correct system order. I found composition useful for assembling features, but the API needs stronger validation to make those combinations reliable as the game grows.

Replication also required more than a matching reader and writer. The same message format carries complete objects and input-only updates, so the receiver needs to know whether an omitted component was removed or simply left out of that message. I had to make that distinction part of the protocol instead of treating every snapshot as a replacement.

Fair measurement needed more than repeated runs. Timing alone does not show that two configurations did the same work, so the benchmark gives both sides of each pair the same input trace at the same simulation ticks, requires every endpoint to finish, and compares the final state with an offline reference. Those checks are what make the timing differences in Section 3 trustworthy.

The fault tests also exposed two limits in our snapshot decoder. A truncated snapshot can change the world before the decoder reports the failure, because decoding is not transactional. A lost delta goes

undetected until the next keyframe, so a one-time change can stay missing for up to 60 updates. The benchmark harness guards against both with checksums and sequence numbers, but the playable network protocol does not.

4.3 The hardest problems, and how I solved them

1. A disconnected server could turn the next request into a crash.

A server disconnect causes a problem beyond the missing reply itself. A client can communicate normally until the server closes and then crash on a later request. A REQ socket expects a reply before the next send. After a timeout, that exchange is still unfinished, so another send can raise an exception. If that exception escapes the network wrapper, it terminates the client.

The socket wrapper therefore catches `zmq::error_t`, records the error and returns failure to the client. The client can then report a disconnection without terminating. The client test stops the server and calls `Tick` three more times, which checks the later requests as well as the initial timeout. That sequence matters because an operation can fail differently after the socket has changed state.

Closing the client needed a similar limit. `Client::Leave` waits at most 250 ms for a reply and avoids repeating the leave request, so a server that has already disappeared does not leave the client waiting through the normal reply timeout. What I took from this was the need to follow an error through the next few operations. Handling the first failure is not enough if the following update can still crash.

2. Ownership had to cover more than movement.

Moving to P2P made the ownership boundary more demanding. Restricting physics to the local runner is not enough because boost timers, countdowns, respawns and restart input also modify player state. A peer could leave a remote runner's position untouched and still change how that runner behaves by expiring its boost or resetting its countdown.

I applied the authority filter to those updates as well as to controllers and physics. The peer test exercises this by giving a remote runner a boost and held restart input, then stepping another player's simulation. It checks position, boost duration, countdown and death count together. This covers the less visible state changes that a movement-only check would miss.

The lesson for me was that ownership has to cover every write to a player's state. When adding an effect, its timers, expiration and cleanup paths deserve the same attention as its movement or animation.

3. Restart and respawn needed different reset rules.

Death and restart both return a runner to a spawn, but I wanted them to have different consequences for the race. Death preserves elapsed time and the latest checkpoint. Restart clears that runner's progress and begins a new countdown. Neither should interrupt the other players.

I kept those transitions separate and reset the state each one actually requires. Respawning clears velocity and platform attachment so the runner does not arrive with its old falling speed or remain linked to a moving platform. Restart also clears race progress, effects and the jump latch. The game tests check these changes alongside the other runner's state, including the case where one player finishes while another continues.

Restart also needs to happen once per press. Calling `Restart` on every tick while R is held would repeatedly reset the countdown to three seconds. I use `restartHeld` to detect the transition, and the test keeps R held for another step to confirm that the countdown continues. This connects the input representation to the intended game action: movement can use a held button, while restart needs an edge.

What I learned from this milestone

I learned that a multiplayer design starts with deciding who can change each part of the game. In client-server mode, one simulation decides the outcome. In P2P, I had to apply ownership rules to timers, effects and resets as well as movement. Sharing the game logic across both modes helped me see exactly which decisions depended on the network architecture. Ownership and reset behaviour should be defined before the network messages that carry them.

The performance results also changed how I judge an optimization. Delta made encoding more expensive but still finished sooner, so CPU cost and message size have to be measured separately before choosing a format. I also learned that repeated runs cannot compensate for unequal workloads. Before trusting a timing difference, I check that each process simulated the same inputs and finished its work.

Appendix A: Screenshots

I took these screenshots while playing in client-server and peer-to-peer modes. Figure A2 is the exception: hidden objects never appear on screen, so it is a map drawn from the course layout.

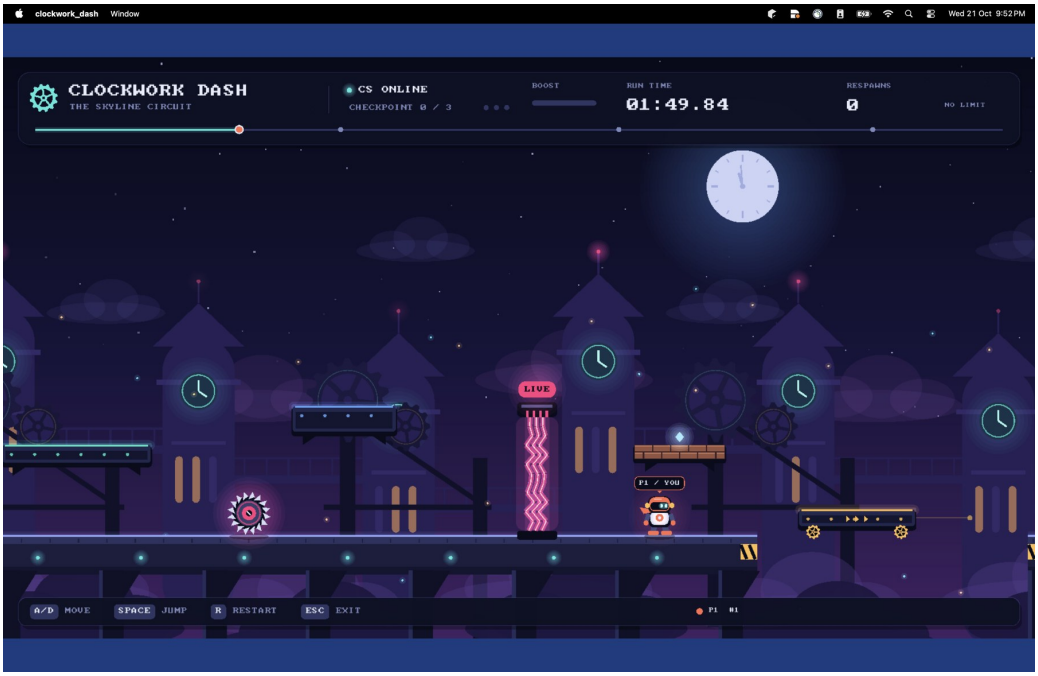


Figure A1. My runner is below a potion ledge in CS mode. The screen also shows static platforms, a moving platform, a saw and a live electric barrier.

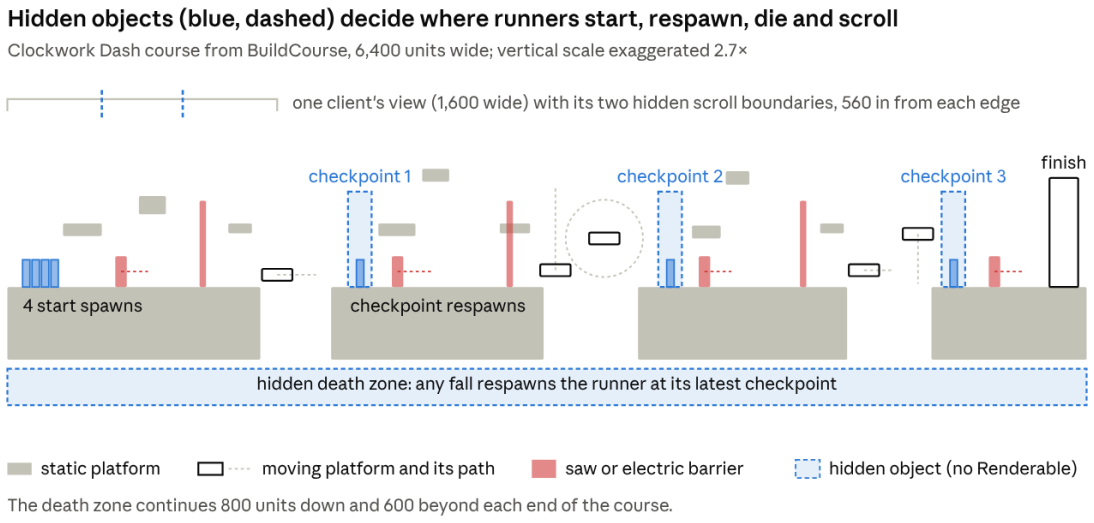


Figure A2. This map shows the full course. The blue outlines mark the starting spawns, checkpoints, respawn locations, death zone and camera boundaries that are hidden during play.

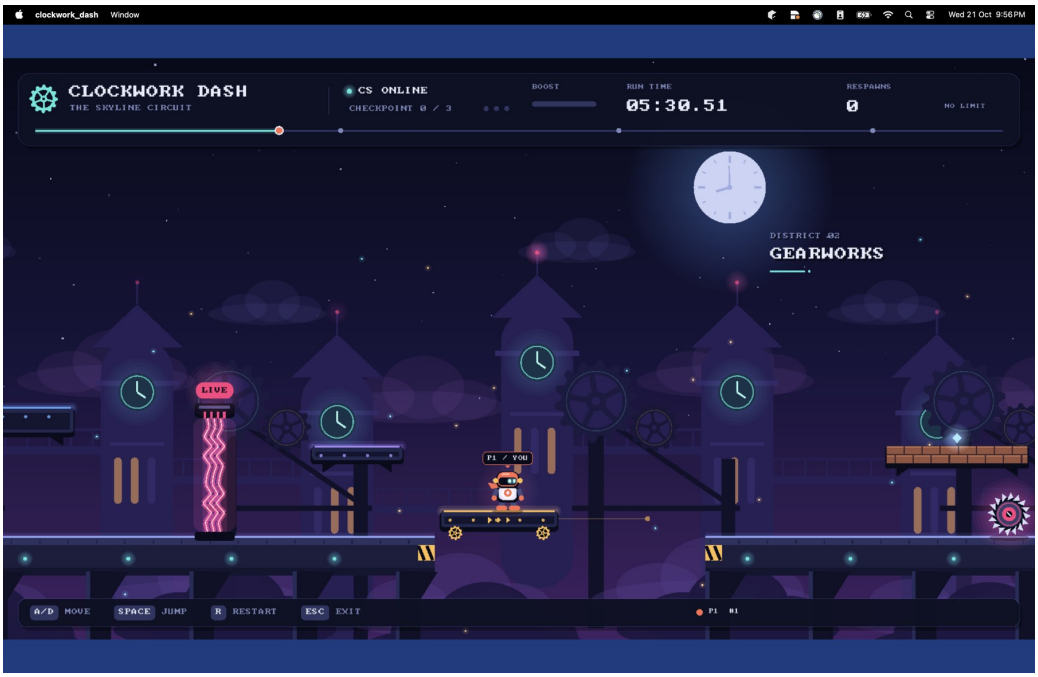


Figure A3. I am standing on the horizontal platform near the left edge of the pit.

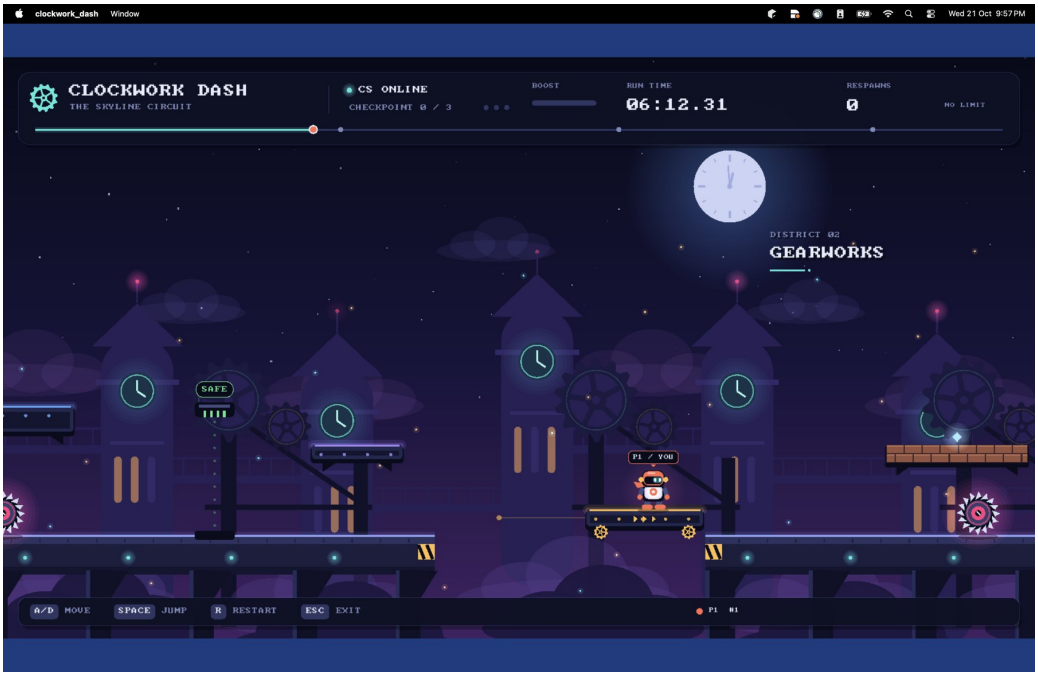


Figure A4. The platform has carried my runner toward the other side. Comparing it with Figure A3 shows the change in position against the same deck edges.

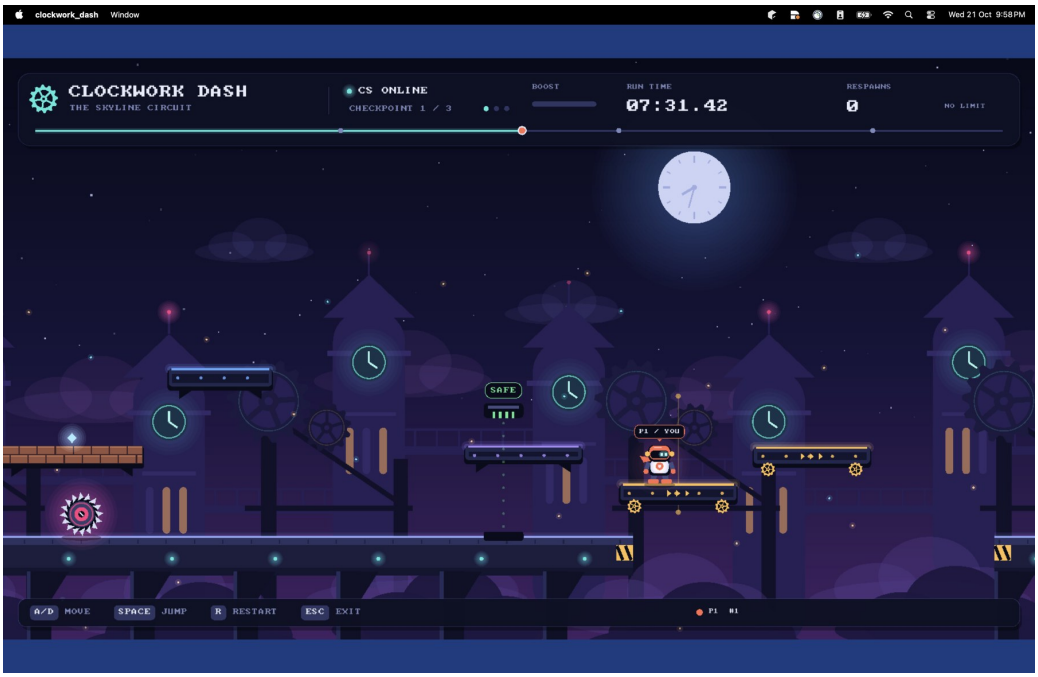


Figure A5. My runner is on the vertical platform after checkpoint 1. The line behind it marks its path up and down.

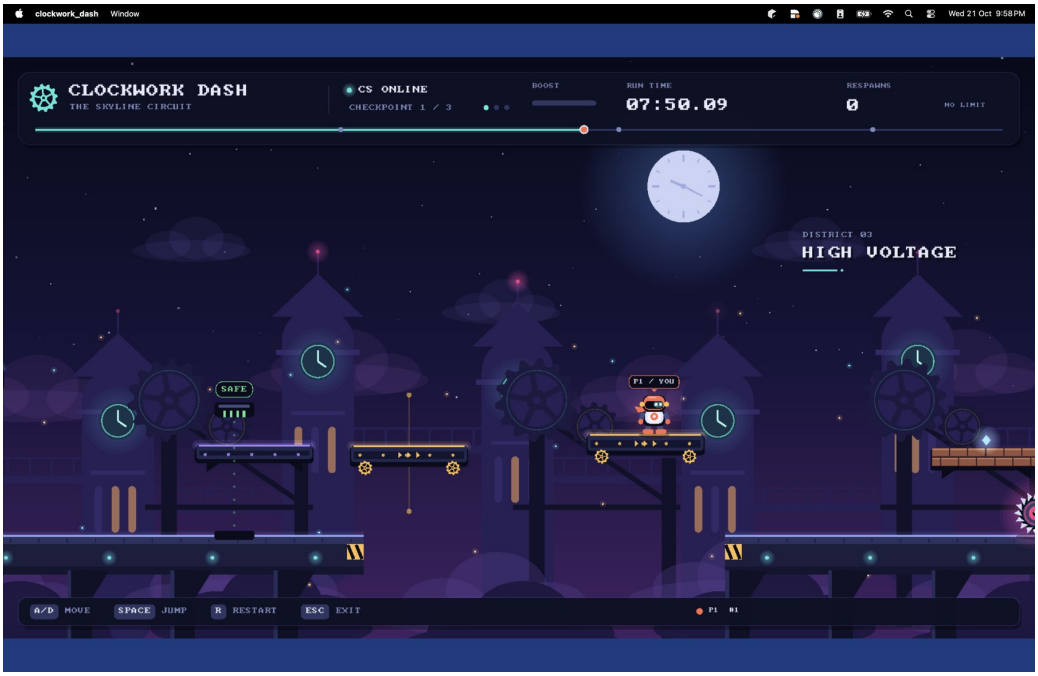


Figure A6. I have reached the circular platform to the right. The vertical platform is still visible on the left, and the camera has scrolled with my runner.

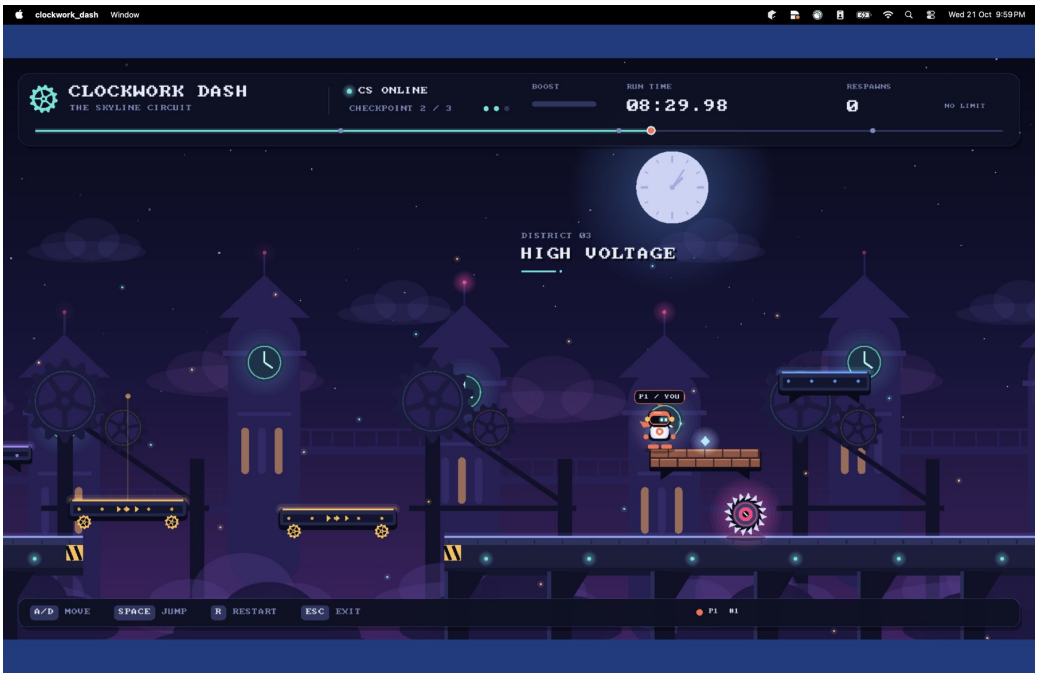


Figure A7. I reached checkpoint 2 without dying. The HUD shows zero respawns and a run time of 08:29.98.

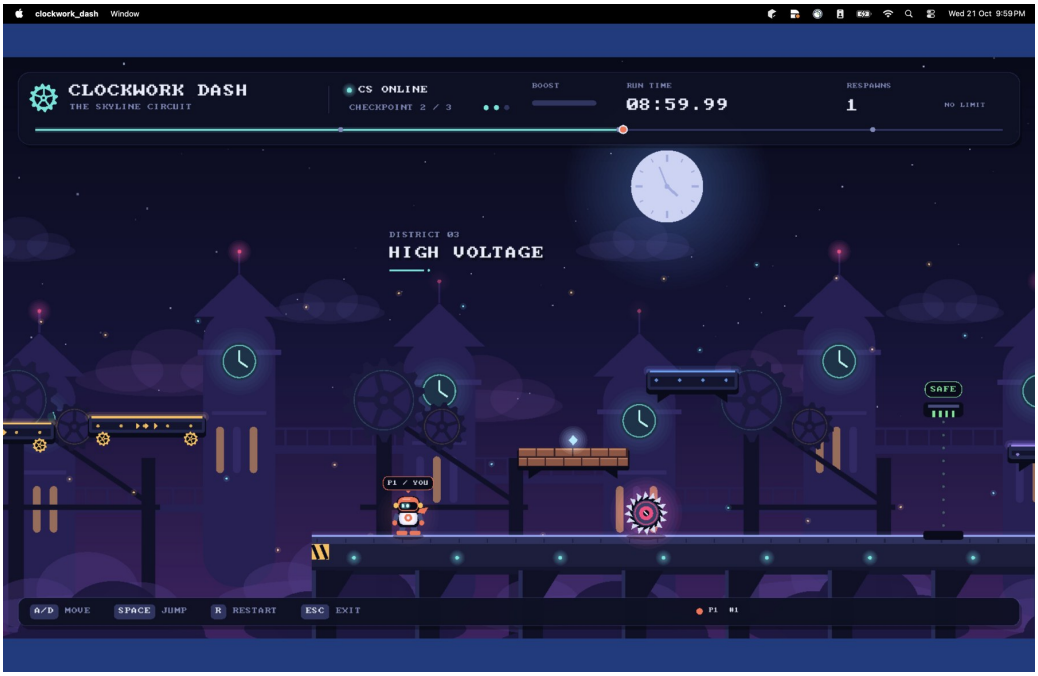


Figure A8. After falling, I returned to checkpoint 2. The respawn count is now one, and the timer has continued to 08:59.99.

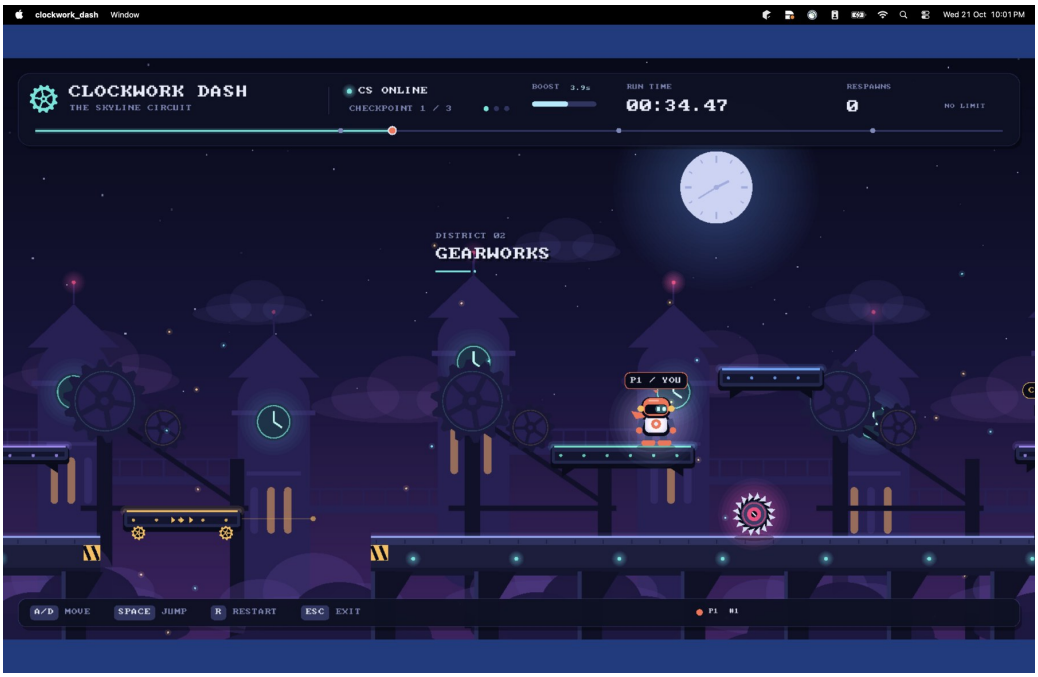


Figure A9. My runner is larger after collecting a potion. The BOOST display shows 3.9 seconds left.

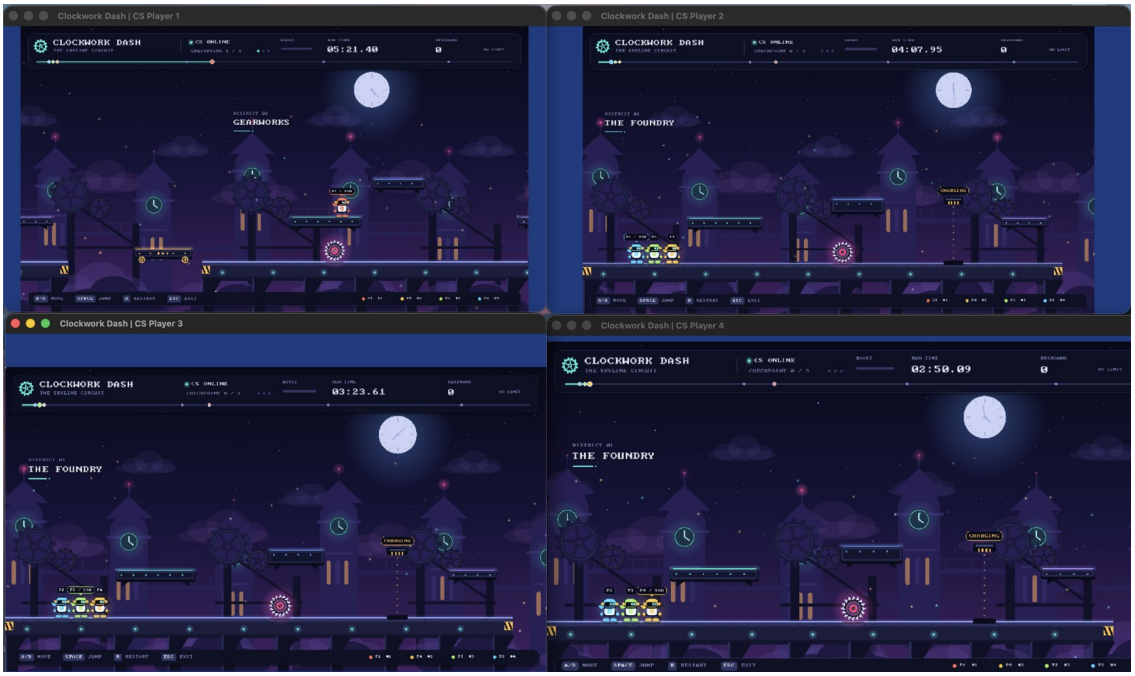


Figure A10. Four clients are connected to the same server. P1 is farther along the course, while P2, P3 and P4 are near the start. Each window follows its own player.

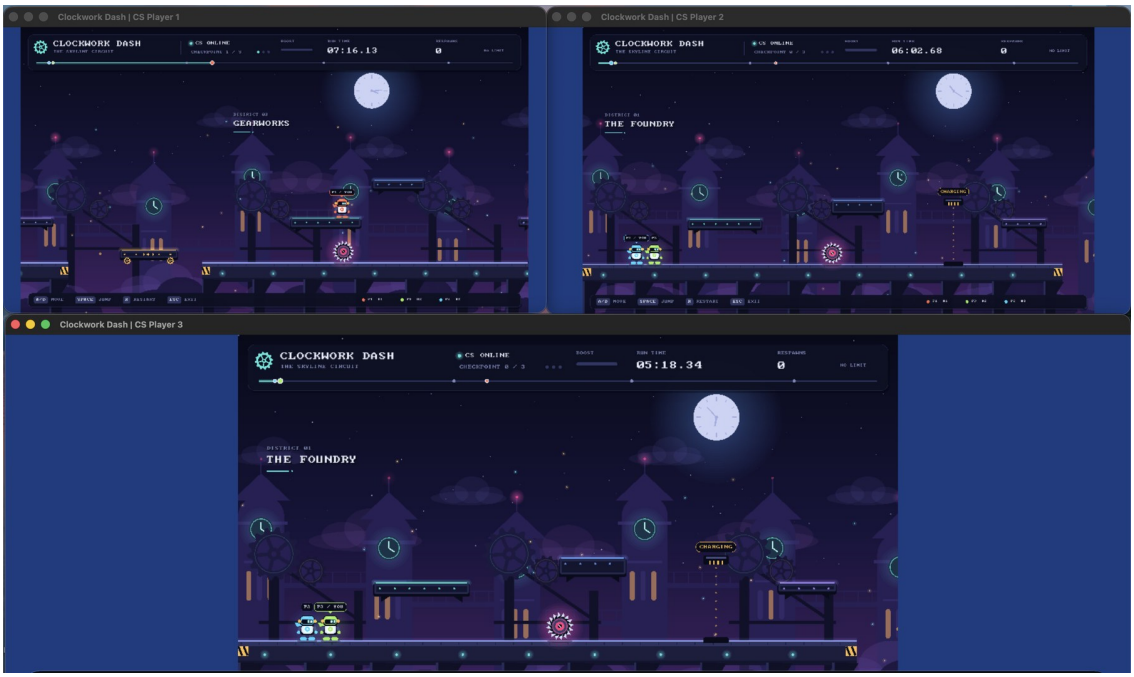


Figure A11. After P4 exits, the other three clients remain connected. P4 is gone from the course and standings, and the remaining timers have kept running.

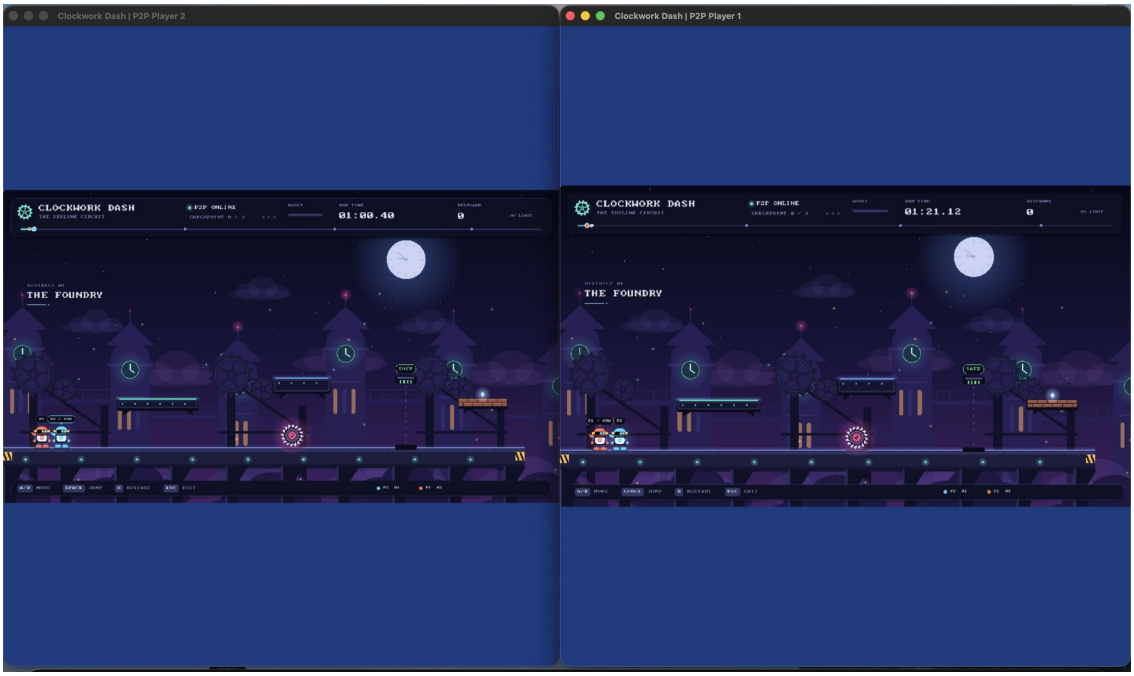


Figure A12. Both P2P windows show both runners and report P2P ONLINE. P2 is controlled from the left window and P1 from the right.

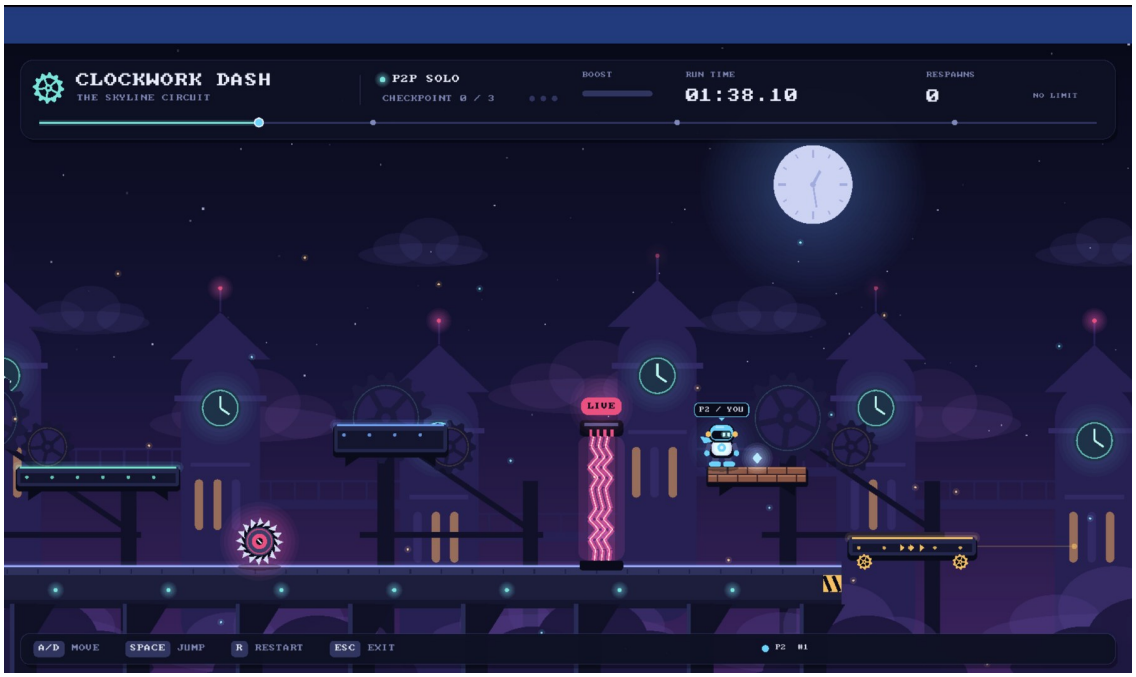


Figure A13. After closing P1, which was peer 0, I continued playing as P2. The status changes to P2P SOLO and only P2 remains in the standings.

Appendix B: Performance graphs

Figures B1–B4 each show one comparison from Section 3. The top row gives completion time for 100,000 ticks and the bottom row gives the total data received by all endpoints. A bar is the mean of ten runs, error bars show one standard deviation, and the percentage under each pair is the change from the first option to the second. Lower is better on every graph. The moving sweep holds 100 static platforms and two players fixed, the static sweep holds 100 moving platforms and two players, and the player sweep uses 100 of each platform type.

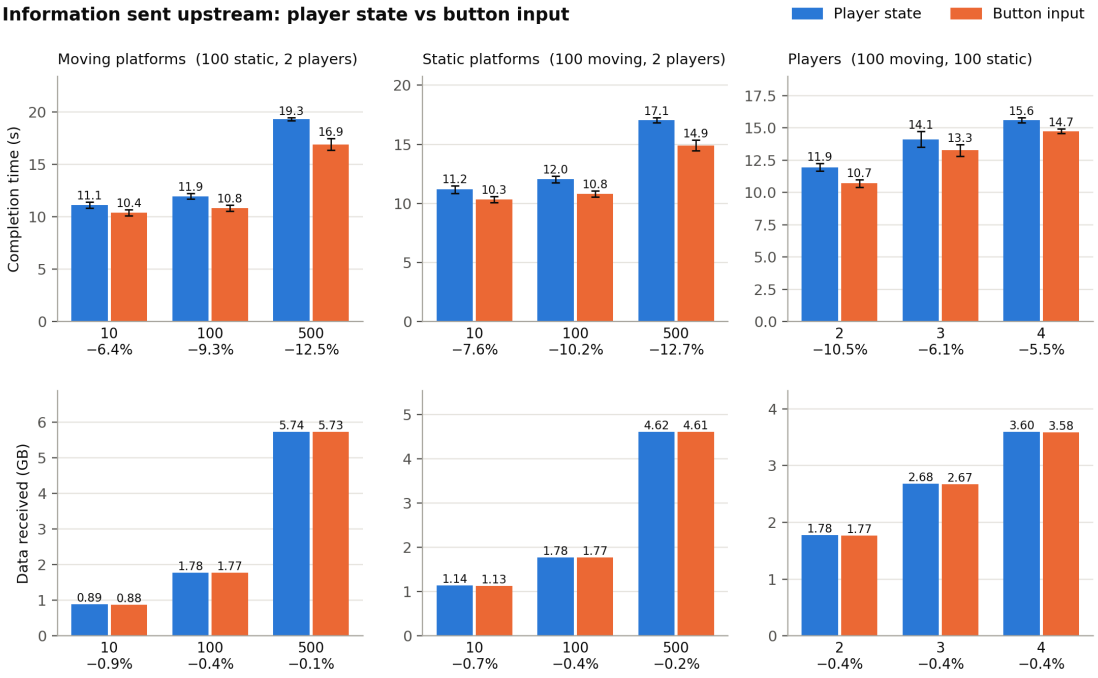


Figure B1. Information sent upstream. Button input finishes 5–13% sooner in every condition, while total data received changes by less than 1% because the downstream world dominates traffic. This comparison also moves player simulation from the clients to the server.

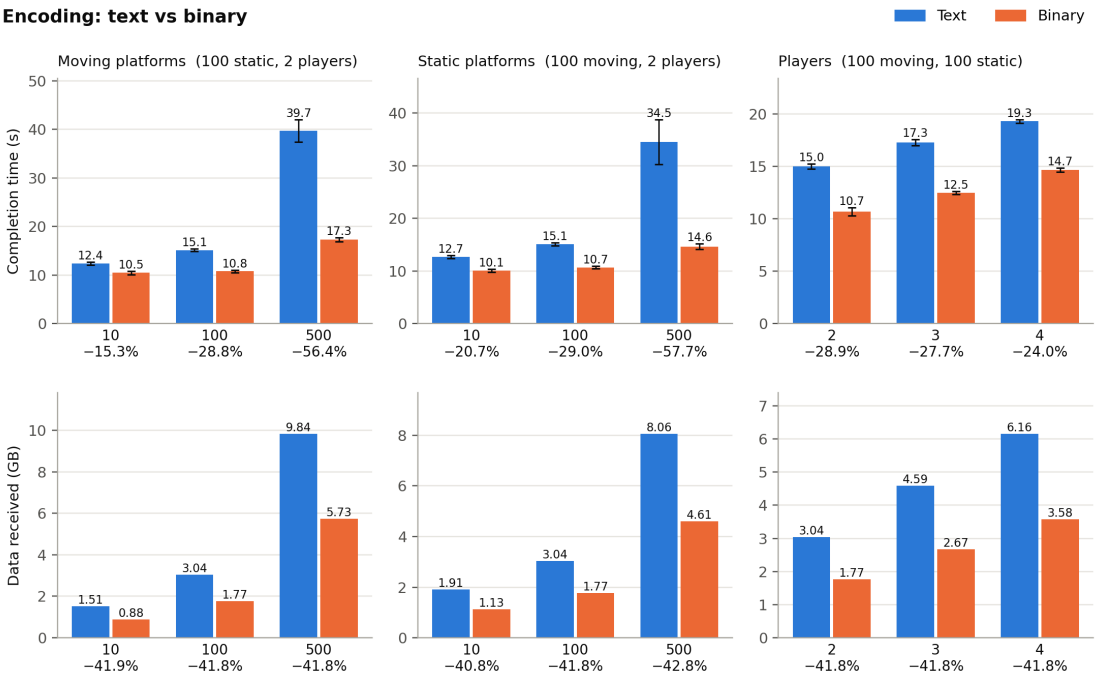
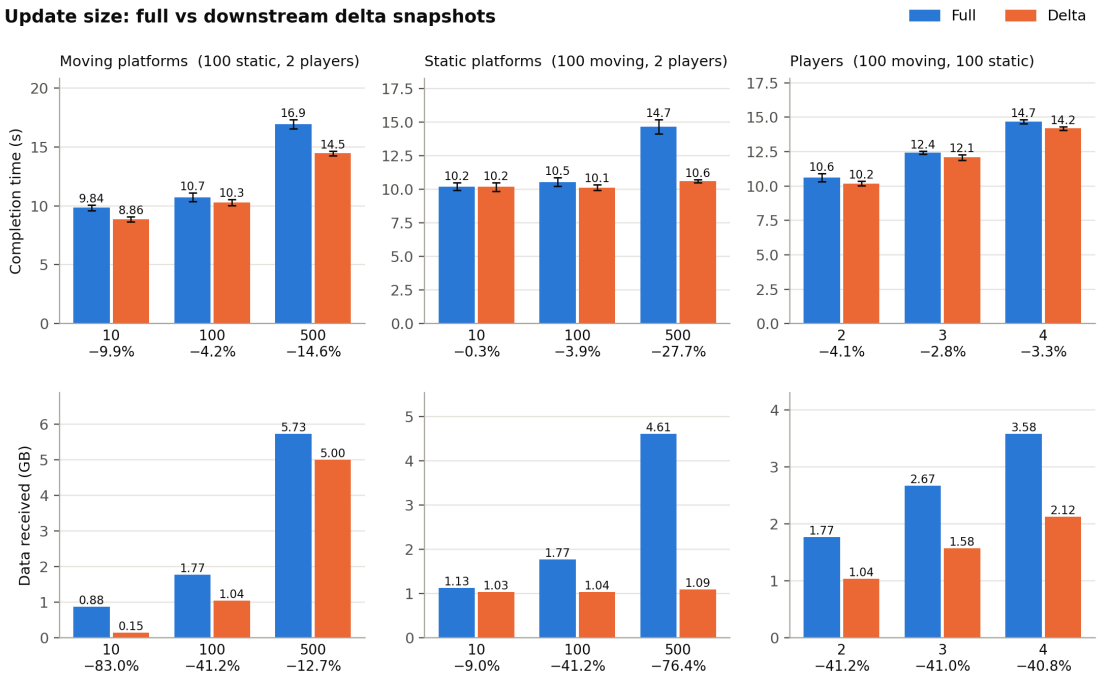
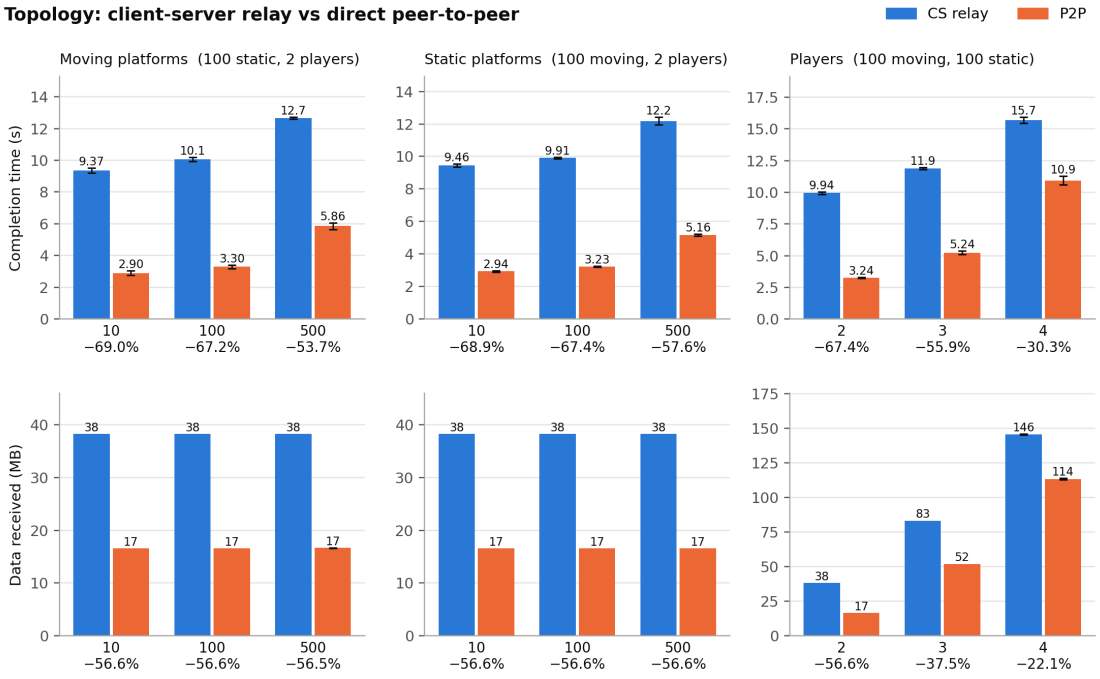


Figure B2. Encoding. Binary finishes 15–57% sooner and receives 41–43% less data in every condition. Its time advantage grows with the number of objects. The large error bars on the 500-object text runs reflect the two groups of times described in Section 3.3.



Mean of 10 runs per bar, error bars ±1 SD. Percent under each pair: change from full to delta. Lower is better.

Figure B3. Update size. Delta receives much less data when most objects are still (83% less with 10 moving and 100 static platforms, 76% less with 500 static) but only 13% less with 500 moving platforms. It finishes sooner in eight of nine conditions, by up to 28%, and ties at 10 static platforms.



Mean of 10 runs per bar, error bars ±1 SD. Percent under each pair: change from CS relay to P2P. Lower is better.

Figure B4. Topology. Direct P2P finishes 30–69% sooner than the CS relay and receives less data. Its lead narrows from 67% at two players to 30% at four, as mesh traffic grows with each added peer.

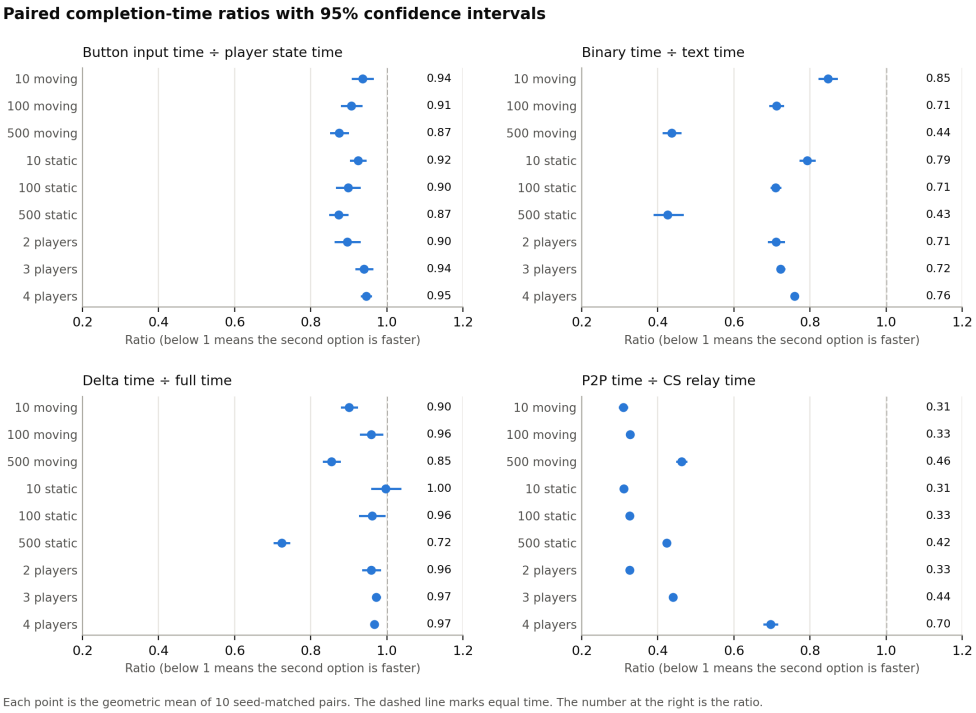


Figure B5. Paired completion-time ratios for all four comparisons, each the geometric mean of ten seed-matched pairs with its 95% confidence interval. Values below 1 favor the second option. Only the delta comparison at 10 static platforms includes 1, which means no measurable difference there.

Appendix C: Running and playing Clockwork Dash

C.1 Building and choosing a mode

Run the following commands from the project root. The build requires CMake, a C++17 compiler, SDL3, SDL3_image, libzmq, cppzmq and pkg-config.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 6
```

Client-server. Start the server first, then run each client command in a separate terminal. The server has no game window. Each client opens a window for one player. Repeat the client command to demonstrate four simultaneous players.

```
./build/clockwork_dash server
./build/clockwork_dash client
./build/clockwork_dash client
```

Peer-to-peer. Start each command in a separate terminal. No server is needed. Add peers with ids 2 and 3 for four players, or omit `--id` to claim a free slot. Each peer can play immediately and shows `P2P ONLINE` after hearing another peer. Closing peer 0 leaves the others running.

```
./build/clockwork_dash peer --id 0
./build/clockwork_dash peer --id 1
```

Multiplayer connects processes on the same computer. The default port base is 8650. If it is occupied, append `--port-base 9000` to every process in that session. Click a game window to control its

player. Release movement keys before switching windows.

C.2 Controls and game rules

Control	Action
A/D or Left/Right arrows.	Move left or right.
Space, W or Up arrow.	Jump.
R.	Restart only your own run.
Esc.	Exit the current game window.

Goal and timing. I designed Clockwork Dash as a time trial across four decks separated by pits. After a personal three-second countdown, pass all three checkpoints in order and enter the finish gate. Checkpoints activate automatically and appear in the HUD rather than as visible markers. Finishing stops your timer. Standings rank finishers by elapsed time, then unfinished players by checkpoint and position. Players can join, finish or restart independently.

Platforms. Static ledges are solid on every side. Jump toward a ledge from the side to land on it. Hitting its underside pushes the runner downward and briefly displaces the ledge, but does not count as a death. Moving platforms can be jumped through from below and carry a runner standing on top. Their horizontal, vertical and circular paths provide routes across the pits, as Figures A3–A6 show.

Hazards and respawning. Falling into a pit, touching a saw or entering a live electric barrier returns the runner to the latest checkpoint, or the starting spawn if no checkpoint has been reached. Respawns are unlimited and the timer keeps running. Saws are always dangerous. Barriers cycle through two seconds SAFE, one second CHARGING and two seconds LIVE. CHARGING is still safe. Respawning grants 0.75 seconds of protection from hazards. Figures A7 and A8 show checkpoint progress being preserved after a fall.

Potions and stomps. Three copper ledges hide potions. Bump one from below to reveal its potion, then touch the potion on top to collect it. Each potion can be collected once per course session. A boost lasts seven seconds, increases the runner's size to 1.25 times normal and ground speed to 1.5 times normal, and leaves jump speed unchanged. Landing on another runner's head makes the attacker bounce and removes the other runner's boost. Otherwise, players pass through one another. Figure A9 shows the enlarged runner and remaining boost time.

Restart and exit. R returns your runner to its starting slot, clears checkpoints, time, respawn count and temporary effects, and starts a new countdown. It does not reset other players or replenish potions. Death, restart and finishing also clear a boost. Esc removes a departing network player while the remaining players continue. A CS race requires its server to stay running. P2P has no designated host.

Network settings. Binary encoding with full snapshots is the default. Append `--encoding text` to select text, or `--delta on` to send changes with periodic full snapshots. Use the same settings on every endpoint. These options change message encoding and update contents while retaining the same game rules.

C.3 How the game meets the required features

I implemented each required game object through the engine's component model. The table connects the assignment requirements to the actual course and describes how to observe each feature. Section 1.3 lists the component combinations in more detail.

Required feature	Implementation and demonstration
A keyboard-controlled player on each client.	Each window controls a distinct robot runner. InputState stores buttons, PlatformerController applies movement and jumping, and RigidBody handles motion. NetIdentity associates the runner with its owner. Open two client or peer windows and move one runner to see its movement in the other view.
At least three different static platforms.	BuildCourse creates nine ledges and four decks. The first three ledges are at (330, 600), (780, 540) and (1310, 600), with sizes 230 × 28, 160 × 40 and 140 × 22. They use different colors, with a copper appearance for the potion ledge. Transform, Renderable and solid Collider components define their geometry, appearance and collision. Jump onto them from the side.
At least two platform movement patterns.	Five platforms follow two horizontal paths, two vertical paths and one circular path. PathMover defines the motion and the physics system carries riders. Stand on a horizontal platform, then a vertical or circular one, to observe two different patterns.
Multiple hidden spawn points.	Four starting slots and three checkpoint destinations use Transform and SpawnPoint without Renderable. Players begin in separate starting slots. Reaching a checkpoint changes the spawn index used for later deaths. The spawn objects themselves are invisible.
A hidden death zone that returns the player to a spawn.	A non-rendered trigger below the course has Transform, Collider and DeathZone. ApplyDeathZones detects a falling runner and PlaceAtSpawn returns it to the assigned spawn, clearing velocity and platform attachment. Fall into a pit after a checkpoint and check that the checkpoint remains while the respawn count increases.
Side scrolling through boundaries or a camera.	Each game window creates a local Camera and two hidden ScrollBoundary objects. UpdateCameras shifts the view when the runner crosses a boundary and clamps it to the course limits. Walking right reveals later parts of the course. Other players retain their own camera views.

The multiplayer implementation also connects these objects across endpoints. In CS mode, the server simulates the world and clients render its updates. In P2P, each peer simulates its runner, calculates platform motion from shared time and exchanges player state and shared interaction events. Networking runs alongside the game loop, and each endpoint stores objects in a World. Four windows demonstrate the complete scene from separate views. Closing one player removes its runner while the others continue. The game supports four P2P slots, while CS admission is not configured with a fixed player limit and the standings display shows up to four racers.

I added checkpoints, timed hazards, potions, stomps and independent race timers to turn the required objects into a playable course. Appendix A illustrates the game, while Section 3 and Appendix B cover the separate CSC 581 performance requirement.